

Código completo

```
# Importa a biblioteca TensorFlow e dá-lhe o nome abreviado "tf"
import tensorflow as tf

# Carrega o conjunto de dados MNIST (imagens de dígitos escritos à mão)
mnist = tf.keras.datasets.mnist

# Divide os dados em treino e teste
# x_train e y_train -> dados e etiquetas para treino
# x_test e y_test -> dados e etiquetas para teste
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normaliza os valores dos píxeis das imagens
# Os valores originais vão de 0 a 255
# Ao dividir por 255.0, ficam entre 0 e 1
# Isto ajuda a rede neuronal a aprender melhor
x_train, x_test = x_train / 255.0, x_test / 255.0

# Cria um modelo sequencial, onde as camadas são adicionadas por ordem
model = tf.keras.models.Sequential([

    # Converte a imagem 28x28 numa única linha com 784 valores
    #  $28 \times 28 = 784$ 
    tf.keras.layers.Flatten(input_shape=(28, 28)),

    # Camada densa (fully connected) com 128 neurónios
    # Usa a função de ativação ReLU
```

```
tf.keras.layers.Dense(128, activation='relu'),

# Camada Dropout que desativa aleatoriamente 20% dos neurónios
# Ajuda a evitar overfitting
tf.keras.layers.Dropout(0.2),

# Camada de saída com 10 neurónios
# Um para cada dígito de 0 a 9
# Softmax converte os valores em probabilidades
tf.keras.layers.Dense(10, activation='softmax')
])

# Configura o modelo para treino
model.compile(

# Define o algoritmo de otimização como Adam
optimizer='adam',

# Define a função de perda para classificação multiclasse
loss='sparse_categorical_crossentropy',

# Define a métrica de avaliação como precisão
metrics=['accuracy']
)

# Treina o modelo com os dados de treino durante 5 épocas
# Uma época = o modelo vê todos os dados uma vez
```

```
model.fit(x_train, y_train, epochs=5)
```

```
# Avalia o desempenho do modelo com os dados de teste
```

```
# Mostra a perda e a precisão final
```

```
model.evaluate(x_test, y_test)
```